

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	S Jefna Princy	Reg. No.:	192572058
Date:	14-08-2026	Duration:	60 minutes

Code of Conduct

I S Jefna Princy- 192572058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

- Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.
---	---	--	---	---

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

CODE REVIEW & REPOSITORY EVALUATION

1. Problem Overview and Repository Organization

1.1 Problem Statement

Urban areas experience increasing noise pollution due to transportation, construction activities, industries, and other human activities. Traditional noise monitoring methods mainly depend on periodic manual measurements and therefore cannot continuously capture changes in urban noise levels. This makes it difficult for authorities to identify persistent noise hotspots and take timely corrective actions.

The proposed **AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System** addresses this problem by using IoT-based noise sensors to collect real-time noise data. The collected data is processed through cloud-based services and analyzed using Artificial Intelligence techniques. GIS technology is used to identify high-noise locations, while an alert mechanism notifies authorized users when noise levels exceed the permitted threshold. A dashboard provides a simple interface for monitoring noise levels, alerts, trends, and analysis results.

1.2 Project Objectives

The major objectives of the project are:

1. To continuously monitor urban noise levels using IoT sensors.
2. To collect and process real-time noise data.
3. To identify high-noise areas using GIS.
4. To analyze historical noise patterns using AI.
5. To predict future noise trends.
6. To generate alerts when noise exceeds permitted limits.
7. To analyze major sources of noise pollution.
8. To generate environmental compliance reports.
9. To support urban planning and environmental decision-making.
10. To improve public health protection and quality of urban life.

1.3 Key Functionalities

Functionality	Description
Noise Data Collection	Collects noise readings from IoT sensors.
Real-Time Monitoring	Displays current noise levels on the dashboard.
AI Analysis	Analyzes historical data and predicts noise trends.
GIS Hotspot Detection	Identifies areas with excessive noise.
Alert Generation	Generates alerts when noise exceeds the threshold.
Noise Source Analysis	Identifies major sources such as traffic and construction.
Report Generation	Generates noise monitoring and compliance reports.
Dashboard	Provides a simple interface for viewing readings and alerts.

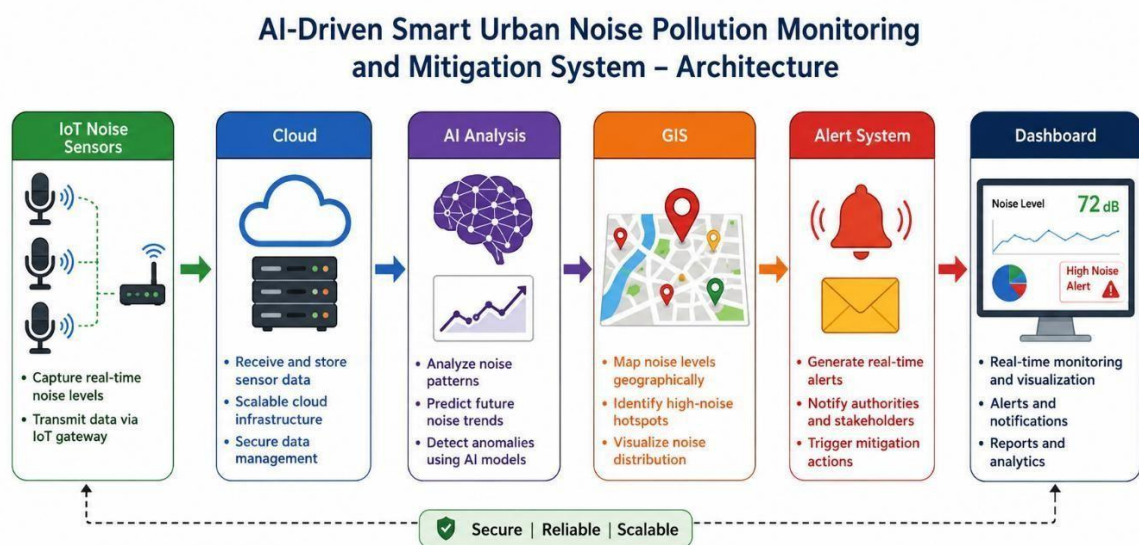
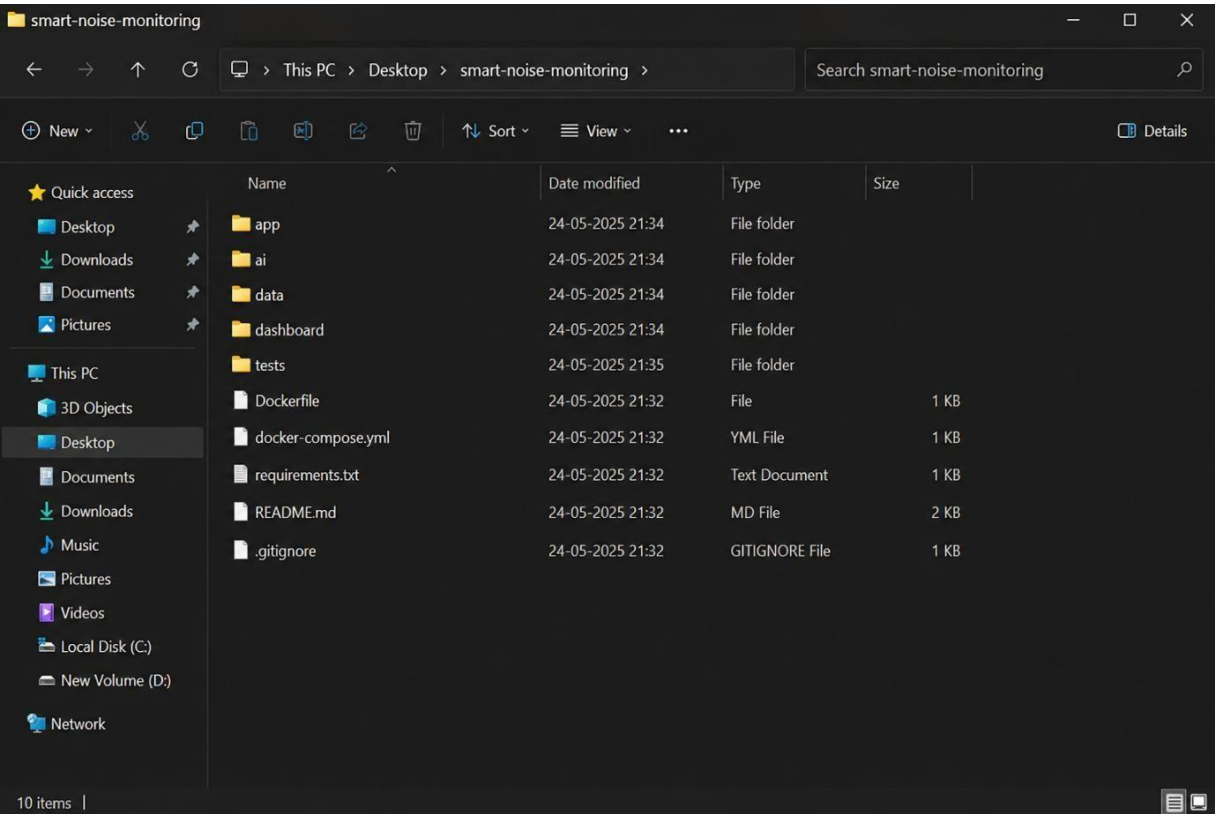


Figure 1: Architecture of the AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System

2. Repository Organization

The project repository is organized into separate folders for application code, AI processing, data, dashboard components, and testing. This structure improves maintainability, readability, collaboration, and future development.

2.1 Repository Structure



2.2 Folder and File Description

Folder/File	Purpose
app/	Contains the main application code.
main.py	Entry point for the application.
api/	Contains API-related components.
models/	Contains data models.
services/	Contains application and processing services.
utils/	Contains reusable utility functions.
ai/	Contains AI-based analysis and prediction modules.
model.py	Defines the AI model.

Folder/File	Purpose
prediction.py	Performs noise prediction.
data/	Stores sample and input data.
dashboard/	Contains the monitoring dashboard.
tests/	Contains test cases.
Dockerfile	Defines the Docker image configuration.
docker-compose.yml	Defines container services and configuration.
requirements.txt	Lists required Python dependencies.
README.md	Provides project information and usage instructions.
.gitignore	Specifies files that should not be tracked by Git.

2.3 Repository Organization Evaluation

The repository follows a modular structure where different responsibilities are separated into appropriate directories. Application logic, AI processing, data, dashboard components, and testing are maintained independently.

This organization makes the project easier to understand and maintain. Developers can modify one component without unnecessarily affecting other components. The structure also supports collaboration because team members can work on different modules independently.

3. Code Quality Review

The source code was reviewed based on **readability, modularity, coding standards, documentation, and maintainability**.

3.1 Readability

The project uses meaningful file and folder names such as prediction.py, model.py, services, and tests. These names clearly indicate the purpose of each component.

The code should use descriptive variable and function names instead of short or confusing names. Proper indentation and consistent formatting should also be maintained.

3.2 Modularity

The project follows a modular structure by separating application logic, AI processing, data handling, dashboard components, and testing.

For example:

- app/ handles application functionality.
- ai/ handles prediction and AI analysis.
- data/ handles sample data.
- dashboard/ handles user interface components.
- tests/ handles testing.

This reduces code duplication and makes individual components easier to modify and test.

3.3 Coding Standards

The project should follow standard Python coding practices such as:

- Meaningful variable and function names.
- Consistent indentation.
- Proper comments where required.
- Functions with single responsibilities.
- Avoiding unnecessary duplicate code.
- Maintaining consistent file organization.
- Following Python naming conventions.

3.4 Documentation

The README.md file provides information about the project, installation requirements, execution procedure, and usage instructions.

Important functions and complex AI-processing sections should also contain suitable comments or documentation so that another developer can understand the implementation.

3.5 Maintainability

The modular repository structure improves maintainability. New sensors, AI models, APIs, and dashboard features can be added without completely redesigning the project.

3.6 Strengths Identified

- Clear repository structure.
- Separation of application and AI components.
- Dedicated testing folder.
- Separate data and dashboard components.
- Docker support for deployment.
- Dependency management through requirements.txt.

- README documentation.

3.7 Areas for Improvement

- Add more detailed documentation for complex functions.
- Increase automated test coverage.
- Add logging for important system events.
- Improve error handling for sensor and network failures.
- Add configuration files for environment-specific settings.
- Use automated code-quality tools such as linters and formatters.

4. Version Control Evaluation

Git is used as the version control system for managing project development.

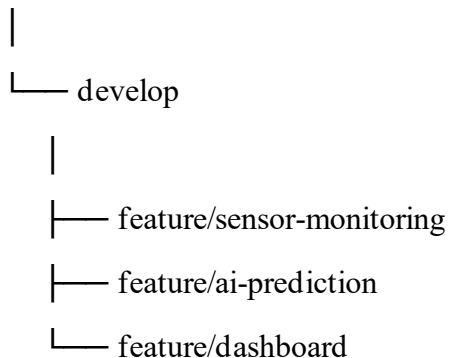
4.1 Git Repository

The project is maintained in a Git repository. Git tracks changes made to source code, configuration files, documentation, and other project components.

4.2 Branching Strategy

A simple branching strategy is used:

main



Main branch:

Contains stable versions of the project.

Develop branch:

Used to combine and test ongoing development.

Feature branches:

Used for developing individual features such as sensor monitoring, AI prediction, and dashboard functionality.

This strategy reduces the possibility of unstable changes directly affecting the main version.

4.3 Commit History

Meaningful commit messages should be used to clearly describe each change.

Examples:

Initial project structure

Add noise sensor data processing

Add AI prediction module

Add dashboard interface

Add test cases

Add Docker configuration

Update README documentation

Meaningful commits make it easier to understand the development history and identify when particular changes were introduced.

4.4 Version Control Workflow

The project follows the workflow:

Create / Modify Code

↓

git status

↓

git add .

↓

git commit

↓

git pull

↓

git merge

↓

git push

This workflow helps developers track modifications, synchronize changes, integrate different branches, and maintain a reliable project history.

4.5 Benefits of Git

Git provides:

- Version tracking.
- Branch-based development.
- Collaboration support.
- Change history.
- Rollback capability.
- Conflict management.
- Safe integration of new features.

5. Code Testing and Validation

Testing is performed to verify that the application functions correctly and handles normal, boundary, invalid, and exceptional conditions.

5.1 Testing Approach

The following testing conditions are considered:

- Normal input.
- Boundary input.
- Invalid input.
- Missing sensor data.
- Excessive noise values.
- Network failure.
- Large sensor input.
- Dashboard functionality.
- AI prediction.
- GIS hotspot detection.

6. Repository Documentation and Maintenance

The repository contains documentation required for installation, execution, and maintenance of the project.

6.1 README Documentation

The README.md file should contain:

1. Project title.
2. Project description.

3. Objectives.
4. System requirements.
5. Installation instructions.
6. Dependency installation.
7. Project execution procedure.
8. Docker execution instructions.
9. Testing instructions.
10. Expected output.
11. Repository structure.

6.2 Dependencies

The requirements.txt file contains the Python packages required by the project.

This makes dependency installation easier and ensures that developers can reproduce the project environment.

6.3 Installation and Usage

The project can be installed by obtaining the repository, installing the required dependencies, and running the application.

Docker configuration further simplifies deployment by providing a consistent execution environment.

6.4 Maintenance

The project can be maintained by:

- Updating dependencies regularly.
- Maintaining meaningful Git commits.
- Adding new test cases.
- Updating documentation.
- Monitoring application logs.
- Improving AI models.
- Adding new sensor support.
- Fixing identified defects.

7. Code Review Findings and Recommendations

7.1 Overall Findings

The review indicates that the project has a **logical repository structure, modular organization, version-control support, testing structure, and deployment configuration.**

The separation of application code, AI modules, data, dashboard, and testing improves maintainability and makes the project easier to extend.

7.2 Recommendations

The following improvements are recommended:

1. Increase automated test coverage.
2. Add detailed API documentation.
3. Implement centralized error handling.
4. Add logging and monitoring.
5. Improve authentication and authorization.
6. Add automated Git-based CI/CD testing.
7. Improve AI prediction accuracy using larger datasets.
8. Add additional GIS visualization features.
9. Improve dashboard usability and accessibility.
10. Regularly update dependencies and security configurations.

7.3 Future Enhancements

Future versions can include:

- More IoT sensors.
- Real-time GIS maps.
- Advanced AI prediction models.
- Mobile application support.
- Automated environmental reports.
- Cloud-based analytics.
- Real-time notifications.
- Historical noise trend visualization.

Final Conclusion

The **AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System** provides a structured approach for monitoring, analyzing, and managing urban noise pollution. The repository is organized into modular components that support maintainability

and collaboration. Git provides effective version control and development tracking, while testing ensures that important system functions are validated.

The use of Docker improves deployment consistency, and proper documentation makes the project easier to install, understand, and maintain. With improvements in automated testing, security, AI prediction, and monitoring, the system can be further enhanced into a reliable smart-city environmental monitoring solution.